

FPGA-TrustZone: Security Extension of TrustZone to FPGA for SoC-FPGA Heterogeneous Architecture

Shupeng Wang^{1*}, Xindong Fan^{2*}, Xiao Xu¹, Shuchen Wang¹, Lei Ju², and Zimeng Zhou^{1,2,3†}

¹School of Cyber Science and Technology, Shandong University

²Quan Cheng Laboratory, Jinan, China

³State Key Laboratory of Cryptography and Digital Economy Security, Shandong University, Qingdao, 266237, China

Abstract—To address the growing security issues faced by ARM-based mobile devices today, TrustZone was adopted to provide a trusted execution environment (TEE) to protect sensitive data. Such TrustZone-based models have been proven to be effective, but they target CPU architectures and do not work for the security of widely used heterogeneous computing platforms such as FPGAs. To solve this issue, we propose a comprehensive SoC-FPGA security framework, FPGA-TrustZone, to support FPGA TEE by extending the security of ARM TrustZone. Experiments on real SoC-FPGA hardware development boards show that FPGA-TrustZone provides high security with low performance overhead.

Index Terms—Trusted execution environment (TEE), SoC-FPGA, ARM TrustZone, security extension

I. INTRODUCTION

In the era of big data, emerging applications like deep learning, intelligent traffic control, and large-scale database management demand information processing capabilities beyond what traditional CPUs can provide. Consequently, new high-performance computing platforms like GPUs, FPGAs, and processing-in-memory chips have emerged [1], [2]. Specifically, FPGAs offer high parallelism, low energy consumption, and programmability [3], making them increasingly popular in the commercial sector. To integrate FPGAs' computing capabilities into traditional CPU-based systems, heterogeneous SoC-FPGA architectures (e.g., Xilinx ZCU102, Xilinx ZCU104, Xilinx Zynq-7010) have been widely deployed in recent years.

Ongoing confidential data leaks have caused significant economic losses and psychological harm, raising widespread concern. At present, data security has become the main concern in the digital age besides performance and energy consumption. Trusted execution environment (TEE) is a trending topic in modern hardware-assisted security architectures for solving data security problems, and it has gained significant attention from both industry and academia. TEEs provide an isolated, protected environment for sensitive data and code, using hardware isolation and encryption to prevent unauthorized access

or tampering. A large number of TEE designs implemented on various instruction set architectures including Intel SGX [4], Keystone [5] and ARM TrustZone [6]. ARM TrustZone offers hardware-level protection and isolation, and it is widely used in embedded systems and mobile devices [7], [8].

However, mainstream TEEs protect only CPUs, leaving heterogeneous computing platforms vulnerable. As FPGA-based heterogeneous platforms become more common, they face increasing attacks. Researchs [9]–[11] have explored extending TEE security features to FPGAs, including hardware isolation and secure environments for third-party applications. Therefore, existing solutions face challenges like insufficient security protection and high overhead. This paper proposes FPGA-TrustZone, extending ARM TrustZone's security features to SoC-FPGA heterogeneous architectures, ensuring FPGA resources are protected. The main contributions of this paper are summarized below:

- We propose a TEE for SoC-FPGA, in which it protects sensitive data and code from external attacks with the TrustZone extension.
- To support security isolation between FPGA programs, a comprehensive FPGA security framework, including the FPGA Secure Monitor, Secure Monitor-Extension and BRAM Protector, is designed.
- We implement a system prototype using an SoC-FPGA board for functional verification and performance analysis. The experimental results demonstrate that the proposed framework can guarantee security while the additional cost is acceptable.

The rest of the paper is organized as follows. Section II introduces the background and the related work for TEEs in FPGA. Section III presents the threat models. Section IV describes the proposed system and design details. Section V shows the experimental setup and evaluations. Section VI concludes this paper.

II. BACKGROUND AND RELATED WORK

ARM TrustZone is an ARM architecture technology designed to enhance security by dividing the system into secure and non-secure execution environments (i.e., TEE and REE), widely used in embedded systems and mobile devices. It enhances system security by isolating external and memory resources at the hardware level. TrustZone virtualizes the CPU

This work is supported by Shandong Provincial Natural Science Foundation (Grant No.ZR2024MF084), National Natural Science Foundation of China (Grant No.62432005), Department of Science & Technology of Shandong Province (Grant No. SYS202201), Quan Cheng Laboratory (Grant No. QCLZD202302), Taishan Scholars Program (No. tsqn202211281).

*Shupeng Wang and Xindong Fan are co-first authors of the article.

†Corresponding author: Zimeng Zhou(Email: zhouzimeng@sdu.edu.cn).

core into secure and non-secure states to manage transactions in the Secure and Normal Worlds separately. A high-permission controller, called the secure monitor, controls the operation of these cores. By integrating software and hardware, ARM TrustZone offers a robust security solution for mobile devices and embedded systems, safeguarding sensitive data and critical tasks from attacks, and enhancing user privacy.

To meet diverse application requirements and evolving market demands, SoC-FPGA heterogeneous platforms have emerged. The SoC-FPGA platform integrates FPGA and CPU on a single chip. The integration of FPGAs with CPUs has unlocked new avenues for innovation in fields like data analytics, machine learning, and embedded systems [12], [13]. This integration offers greater flexibility, configurability, and performance across various applications. However, SoC-FPGA also brings additional security risks. Thus, protecting the security of the SoC-FPGA heterogeneous architecture is crucial.

Yanling Wang et al. [14] propose a scheme called RCTEE, offering remote users a runtime-customizable TEE on a cloud SoC-FPGA. This scheme supports secure and dynamic IP deployment and invocation. SGX-FPGA [11] proposes a framework that connects SGX enclaves and FPGA, creating a secure hardware isolation path between the CPU and FPGA to ensure the confidentiality and integrity of heterogeneous CPU-FPGA systems. In SoC-FPGA architecture, Embassy [9] uses an FPGA terminal to build a secondary TEE for third-party Trusted Applications, extending TrustZone's use. This not only shields trusted applications in the secondary TEE from the Normal World but also protects both secure and normal operating systems from malicious third-party applications. However, the secondary TEE does not address the trusted boot of the FPGA. TEEOD [10] proposes isolation tools to create secure environments for common applications and creates a secure processor for each isolated environment. However, there are different problems such as high overhead, high communication cost, insufficient external storage security and so on when the existing schemes are really applied to the actual applications. This is the challenge we strive to address in this paper.

III. THREAT MODEL

In this work, we target an emerging new system architecture, the SoC-FPGA heterogeneous architecture, which does not have a commonly adopted threat model definition in the community. The integration of FPGAs in SoCs introduces new attack surfaces that need to be considered in TEE systems. These attack surfaces can be categorized into four aspects:

1) *Internal attacks on the FPGA*. Modern FPGAs support concurrent execution for multiple tasks. The concurrent execution introduces a risk of stealing confidential models or data by leveraging internal FPGA resources. And attackers embed malicious logic into FPGA designs, such as through third-party IP or unverified code modules, to disrupt system functionality, steal or leak sensitive data [14].

2) *Using a compromised FPGA to attack CPU-side resources*. FPGAs in mobile devices typically have access per-

missions to the CPU-side memory, which contains sensitive data. Then attackers can inject malicious logic into the FPGA-side, directly accessing the memory or registers of the CPU-side, thereby stealing or tampering with sensitive data, which is more likely to cause system crashes.

3) *Invaded CPU attacks the FPGA*. Attackers can control the Normal World on the CPU-side through malicious code injection, thereby achieving deep penetration into the entire system, directly affecting the security of the system [10]. Then the invaded CPU can exploit its control over resources to attack the FPGA through malicious code, communication protocol vulnerabilities, etc., illegally accessing FPGA-side resources, changing logic functions, stealing sensitive user data, and disrupting system integrity and user data confidentiality.

4) *External attacks between CPUs and FPGAs*. In the SoC-FPGA system architecture, the AXI bus serves as a critical communication channel between CPUs and FPGAs. This interface plays an important role in coordinating complex operations, but it introduces new vulnerabilities in data security such as corrupting communication signals and response signals [15]. And attackers can use various means to monitor, interfere with, and steal user data transmitted on the AXI bus.

IV. SYSTEM DESIGN

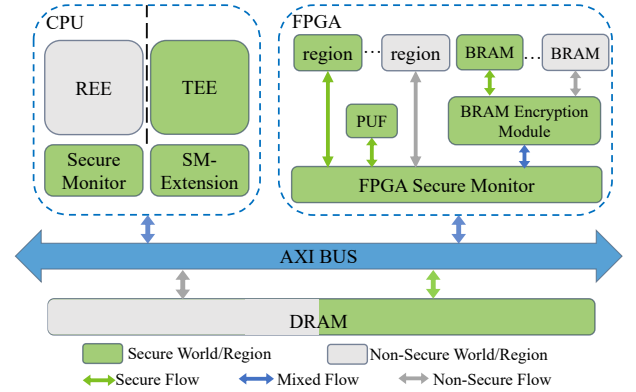


Fig. 1. Overall Framework of FPGA-TrustZone.

We propose FPGA-TrustZone, which builds FPGA TEE by extending the security features of ARM TrustZone to the SoC-FPGA platform. As shown in Fig. 1, FPGA-TrustZone consists of three essential components: *FPGA Secure Monitor (FPGA-SM)*, *Secure Monitor-Extension (SM-Extension)*, and *BRAM protector*. We design the *FPGA-SM* to provide a lightweight inner resource isolation mechanism for the FPGA by extending the security features of ARM TrustZone. The *SM-Extension* working on the CPU-side is designed to cooperate with the work of the *FPGA-SM*. *BRAM protector* utilizes the constructed physical unclonable function (PUF) to support the *BRAM Encryption Module* to encrypt the BRAM to protect sensitive information from unauthorized access and tampering.

A. Challenges of Protecting Soc-FPGA

To develop the FPGA-TrustZone for the secure SoC-FPGA heterogeneous architecture, we have three challenges to overcome:

is the same as that of the target region. If the security level is acceptable and the target region is secure, *Transaction monitor* continues to check whether the DeviceID carried by the transaction is in the device access list of the target region. If all the checks pass, the transaction is forwarded to the target region, else the *Transaction monitor* poisons the transaction and sends an interrupt request (IRQ) to the platform management unit (PMU). Specifically, *Transaction monitor* has two poison methods: 1) Return an error code (default). 2) Modify the destination address of the transaction so that it redirects to an empty area.

C. Secure Monitor-Extension

The implementation of the FPGA-SM is inseparable from the help of the CPU-side, i.e., the *SM-Extension*, which provides the security features of ARM TrustZone, the dynamic configuration of application security requirements, and the handling of secure transmission. *SM-Extension* is the extension of Secure Monitor, the core component of ARM TrustZone, to implement the secure communication between CPU and FPGA. It can ensure the security and credibility of the system.

Dynamically configure. *SM-Extension* proposes multiple interfaces for applications. An application can use these interfaces to create, configure, use, and destroy a FPGA region and it can control the entire life cycle of the region.

Secure Transmission. *SM-Extension* is able to handle secure and non-secure transactions from FPGA to prevent potential security threats and attacks. It is used for encrypting and decrypting transactions transmitted between CPU and FPGA, to ensure the security of data transmission, and minimize the additional overhead of performance. It uses the symmetric key algorithm AES-128bit with Electronic CodeBook mode. The key is stored in the *SM-Extension* and *FPGA-SM* through hardcoding. It can also handle transactions from FPGA. After the FPGA sends the transaction to CPU, *SM-Extension* first checks its security level, then decrypts the data with the corresponding key and forwards the data to the corresponding applications.

D. BRAM Protector

In order to prevent the leakage of BRAM by malicious means, FPGA-TrustZone proposes *BRAM Protector* to protect sensitive information. *BRAM Protector* implements the *PUF* to generate the Initialization Vector (IV) values of the AES-CBC algorithm to ensure the uniqueness and randomness of the IV. Then *BRAM Protector* can encrypt BRAM to protect sensitive information in the system from unauthorized access and tampering.

The PUF is a security feature based on physical properties that generate unique identifiers using small differences in chip manufacturing processes. The PUF can be used for various security applications, including device authentication and key generation, as well as other security fields, providing reliable solutions for various security applications. Here, PUF is used to generate IV values in the AES-CBC algorithm. The IV generated by the PUF can satisfy uniqueness and

randomness well. Our design combines Arbiter PUF and RO PUF, including ring oscillators, latches, and arbitrators. The arbiter is responsible for controlling the operational status of the ring oscillator and latch, while the ring oscillator and latch are responsible for generating the output data of the PUF.

BRAM Encryption Module is used to encrypt BRAM. The structure is shown in the Fig. 3. In order to reduce the clock overhead of BRAM storage and ensure the security of sensitive data, *BRAM Encryption Module* provides optional data encryption services. Regions can use the IS_ENCRY register to indicate whether the data transaction needs to be encrypted. Each transaction has an additional IS-ENCRY bit to indicate whether the transaction needs to be encrypted. If IS_ENCRY=1, encryption or decryption is required. If IS_ENCRY = 1, it needs to be encrypted. We add a *BRAM Encryption Module* between the BRAM and regions. When the region does not enable memory encryption, data from the region to BRAM will not have any additional delays.

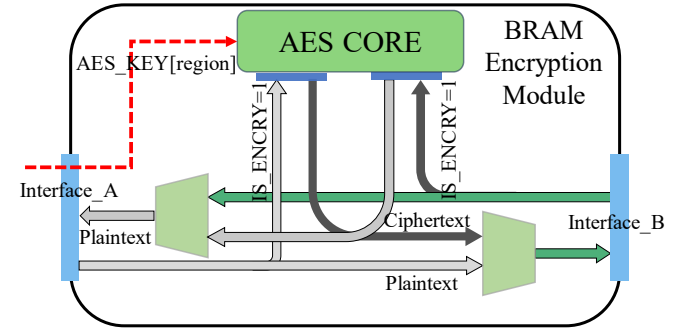


Fig. 3. BRAM Encryption Module.

BRAM Encryption Module uses the AES encryption core with a cipher block chaining (CBC) mode for encryption. Additionally, we consider the security of AES keys. FPGA-TrustZone stores AES keys in FPGA-SM, keeping them transparent to regions. Regions cannot see ciphertext and can only access plaintext decrypted by the *BRAM Encryption Module*, making it difficult to derive the key from plaintext alone. Encrypting BRAM data enhances confidentiality and security, protecting against physical and logical attacks. This ensures the storage security of the FPGA.

E. Data Flow Analysis

The FPGA-TrustZone security architecture encompasses both boot-time and runtime security. Boot-time security is ensured by ARM TrustZone and the FPGA trusted boot process. Runtime security relies on system components to protect programs and data from leakage or tampering. Fig. 4 provides a detailed overview of the data flow in the FPGA-TrustZone architecture. The PMU continuously monitors the system for abnormal behavior and illegal operations during data transmission.

During interaction between CPU applications and FPGA regions, secure-sensitive data is first encrypted by *SM-Extension*, ensuring it remains in ciphertext form during transmission, thereby enhancing resistance to side-channel attacks. Upon

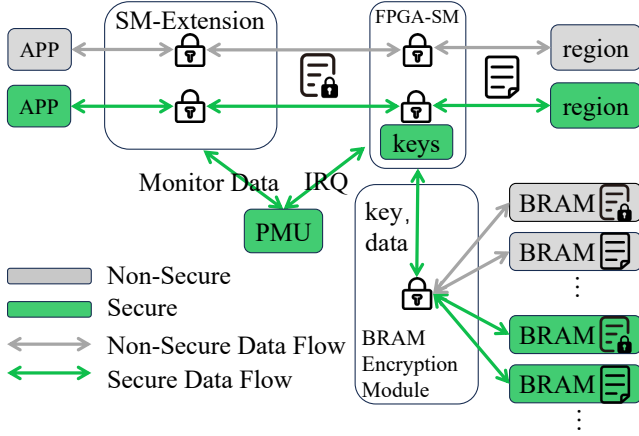


Fig. 4. Secure Data Flow.

TABLE I
ANALYSIS OF SECURITY ABILITIES OF THREE ARCHITECTURES

Ability	FPGA-TrustZone	SGX-FPGA	TPM-TrustZone	PUF-TrustZone
CPU Secure Boot	✓	✓	✓	✓
FPGA Secure Boot	✓	✓	✓	✗
BRAM Encryption	✓	✓	✓	✗
Monitoring Access	✓	✓	✗	✗
PMU IRQ Analysis	✓	✗	✗	✗
Poison Illegal Access	✓	✗	✗	✗
Cold-boot Attack Resistant	✓	✗	✓	✓

data arrival, *FPGA-SM* performs a legitimacy check on the transmission transaction. If the transaction is illegal, *FPGA-SM* sends an IRQ to the PMU and poisons the transaction. If the transaction is legal, *FPGA-SM* decrypts the data and transfers it to the corresponding region for processing. After processing, *FPGA-SM* encrypts the results and sends them to the CPU's *SM-Extension* for decryption and return to the application. Additionally, the region can use the *BRAM Encryption Module* to encrypt its BRAM. This allows sensitive data to be stored in encrypted form in BRAM, further enhancing system security.

V. EXPERIMENTAL EVALUATION

We implement a prototype system for the FPGA-TrustZone security framework on the Zynq UltraScale+XCZU9EG-2FFVB1156E MPSoC (ZCU102). Based on the prototype system, this section conducts two types of experiments to comprehensively evaluate the effectiveness and performance of FPGA-TrustZone: 1) security evaluation, in which we specifically analyzed how the security function implemented by FPGA-TrustZone is to resist the threat model proposed in Section III; 2) performance analysis, in which we analyze the cost of resources and time of FPGA-TrustZone.

A. Security Evaluation

The security of FPGA-TrustZone is verified through various situations, various applications and various illegal means to

TABLE II
RESOURCE USAGE OF THE INITIAL HARDWARE DESIGN, PUF, BRAM ENCRYPTION MODULE AND FPGA-SM.

Component	LUT	FF	LUTRAM	BUFG	BRAM
PUF	96 0.04%	292 0.05%	0 0%	0 0%	0 0%
BRAM Encryption Module	6765 2.47%	4939 0.90%	0 0%	4 0.99%	0 0%
FPGA-SM	3435 1.25%	2046 0.37%	0 0%	0 0%	0 0%
The Initial Hardware Design	24321 8.87%	25595 4.67%	3098 2.15%	6 1.49%	4 0.44%

access the secure and non-secure regions. And we compare the security of FPGA-TrustZone with other secure FPGA architectures including SGX-FPGA, TPM-TrustZone [16] and PUF-TrustZone [17] in Table I. FPGA-TrustZone demonstrates prominent advantages.

Moreover, FPGA-TrustZone can effectively prevent the threat model proposed in Section III: 1) *Internal attacks on the FPGA*: FPGA-TrustZone divides the FPGA-side into secure and non-secure areas, providing logical resource isolation and preventing malicious IPs from unauthorized access. And, FPGA-TrustZone implements a trusted boot process, ensuring system integrity and security. Data stored in external memory is encrypted, ensuring it exists as ciphertext. Even if attackers obtain the ciphertext, recovering the plaintext is difficult, so as to protect firmware code and key data from leakage or theft. Encrypting the Bitstream ensures data confidentiality during transmission and storage, thereby preventing tampering during transmission.

2) *Using a compromised FPGA to attack CPU-side resources*: FPGA-TrustZone resists attacks on the CPU-side by using ARM TrustZone to secure sensitive data and critical operations within a protected environment. Additionally, *SM-Extension* and *FPGA-SM* control FPGA access to the CPU, preventing threats by monitoring and verifying access transactions.

3) *Invaded CPU attacks the FPGA*: The *FPGA-SM* strictly controls CPU access, preventing unauthorized users from tampering with the FPGA security area and protecting the system from potential harm.

4) *External attacks between CPUs and FPGAs*: The encrypted data transmission can effectively prevent attacks such as eavesdropping and tampering, thereby ensuring data confidentiality.

B. Performance Evaluation

Then we measure the performance overhead of FPGA-TrustZone on the ZCU102 development board. The main hardware resources used on devices are as follows:

- 1) LookUp Table (LUT) and Flip-Flop (FF), used to implement logical functions.
- 2) LookUp Table RAM (LUTRAM) and BRAM, used to store data.
- 3) Global clock buffer (BUFG), used for clock signal management, allocation, and processing.

TABLE III
TIME OVERHEAD OF CRITICAL OPERATIONS.

Operation	Time Overhead
IV_generation	0.5 clock cycles
AES_CBC Algorithm	14.5 clock cycles
Memories access	2 ms
Poisoning illegal access	202 ms

TABLE IV
PERFORMANCE EVALUATION.

Benchmarks	Baseline	FPGA-TrustZone	Additional Overhead
Vadd	92439ns	113037ns	22.28%
Matmul	11665ns	13834ns	18.59%
Matmul: partition	1706ns	2042ns	19.70%

Firstly, we evaluate the resource overhead of FPGA-TrustZone. PUF, BRAM Encryption Module and FPGA secure monitor as the three major components. Here, *Used* represents the hardware resources used in the designed solution, while *Available* means the total number of configurable resources for ZCU102 development board. Then $Utilization\ Ratio = Used / Available$.

We evaluate the resource usage of three main components and the initial hardware design on the ZCU102 development board. Table II presents the hardware resource utilization. Results indicate that the resource cost brought by each component no more than 3% and the overall resource consumption does not exceed 9%. Users can customize hardware designs as needed, provided resource usage stays within available limits. Since the FPGA-TrustZone framework maintains a constant footprint across different SoC-FPGAs, its relative resource usage decreases on platforms with larger capacities.

In addition, we evaluate the time cost of part key operations related to the FPGA-side, including the BRAM encryption module's time consumption, illegal access poisoning time, and memory access operation time. Table III shows the results.

We test runtime performance using three Xilinx benchmark programs: vector addition, matrix multiplication, and matrix multiplication (partition). Table IV presents a comparison between our solution and an unsecured SoC-FPGA. Results show that FPGA-TrustZone introduces a 22.28% overhead for basic operations, which is acceptable given its protection of data storage, computation, and transmission security. Additionally, FPGA-TrustZone incurs an 18.59% overhead for parallel computation, indicating its suitability for parallel execution. The overhead introduced while protecting the overall security of the SoC-FPGA architecture is acceptable.

In summary, FPGA-TrustZone is feasible in terms of overhead on the ZCU102 development board.

VI. CONCLUSION

This paper proposes FPGA-TrustZone, a solution that extends Arm TrustZone's security features to the FPGA, achieving hardware resource isolation and protecting sensitive data in Soc-FPGA architecture. FPGA-TrustZone introduces three important components: FPGA Secure Monitor, Secure Monitor-

Extension, and BRAM Protector. FPGA-TrustZone monitors transactions in real-time using Secure Monitor-Extension and FPGA Secure Monitor, verifying the legitimacy of access transactions and poisoning unauthorized access. This approach effectively enhances the overall security of the system. In addition, to further ensure the security of data, FPGA-TrustZone introduces BRAM Protector for encrypting data stored in BRAM. Our implementation and evaluation of actual hardware indicate that FPGA-TrustZone provides relatively low resource and performance overhead while ensuring security, providing a solid guarantee for system security and performance.

REFERENCES

- [1] B. Li, B. Yan, and H. Li, "An overview of in-memory processing with emerging non-volatile memory for data-intensive applications," in *Proceedings of the 2019 on Great Lakes Symposium on VLSI*, 2019, pp. 381–386.
- [2] F. Liu, Y. Liang, and L. Wang, "A survey of the heterogeneous computing platform and related technologies," *Proc. DEStech Trans. Eng. Technol. Research IMEIA*, vol. 1, pp. 6–12, 2016.
- [3] J. Hoozemans, J. Peltenburg, F. Nonnemacher, A. Hadnagy, Z. Al-Ars, and H. P. Hofstee, "Fpga acceleration for big data analytics: Challenges and opportunities," *IEEE Circuits and Systems Magazine*, vol. 21, no. 2, pp. 30–47, 2021.
- [4] V. Costan and S. Devadas, "Intel sgx explained," *Cryptology ePrint Archive*, 2016.
- [5] D. Lee, D. Kohlbrenner, S. Shinde, K. Asanović, and D. Song, "Keystone: an open framework for architecting trusted execution environments," Association for Computing Machinery, 2020.
- [6] S. Pinto and N. Santos, "Demystifying arm trustzone: A comprehensive survey," *ACM computing surveys (CSUR)*, vol. 51, no. 6, pp. 1–36, 2019.
- [7] T. Sun, B. Li, Y. Teng, Y. Gao, and W. Dong, "dtee: A declarative approach to secure iot applications using trustzone," in *2024 23rd ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN)*. IEEE, 2024, pp. 200–212.
- [8] X. Xie, H. Wang, Z. Jian, T. Li, W. Wang, Z. Xu, and G. Wang, "Memory-efficient and secure dnn inference on trustzone-enabled consumer iot devices," *arXiv preprint arXiv:2403.12568*, 2024.
- [9] D. Hwang, S. Yeleuov, J. Seo, M. Chung, H. Moon, and Y. Paek, "Ambassy: A runtime framework to delegate trusted applications in an arm/fpga hybrid system," *IEEE Transactions on Mobile Computing*, 2021.
- [10] S. Pereira, D. Cerdeira, C. Rodrigues, and S. Pinto, "Towards a trusted execution environment via reconfigurable fpga," *arXiv preprint arXiv:2107.03781*, 2021.
- [11] K. Xia, Y. Luo, X. Xu, and S. Wei, "Sgx-fpga: Trusted execution environment for cpu-fpga heterogeneous architecture," in *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2021, pp. 301–306.
- [12] Y.-C. Lin, B. Zhang, and V. Prasanna, "Hitgnn: High-throughput gnn training framework on cpu+ multi-fpga heterogeneous platform," *IEEE Transactions on Parallel and Distributed Systems*, 2024.
- [13] Y. Sun, H. Liu, X. Liao, H. Jin, and Y. Zhang, "Fpga-based acceleration architecture for apache spark operators," *CCF Transactions on High Performance Computing*, pp. 1–14, 2023.
- [14] Y. Wang, X. Chang, H. Zhu, J. Wang, Y. Gong, and L. Li, "Towards runtime customizable trusted execution environment on fpga-soc," *arXiv preprint arXiv:2307.04375*, 2023.
- [15] E. M. Benhani, L. Bossuet, and A. Aubert, "The security of arm trustzone in a fpga-based soc," *IEEE Transactions on Computers*, vol. 68, no. 8, pp. 1238–1248, 2019.
- [16] M. Gross, K. Hohentanner, S. Wiehler, and G. Sigl, "Enhancing the security of fpga-socs via the usage of arm trustzone and a hybrid-tpm," *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, vol. 15, no. 1, pp. 1–26, 2021.
- [17] C. Aitchison, R. Buckle, A. Ch'ng, C. Clarke, J. Malley, and B. Halak, "On the integration of physically unclonable functions into arm trustzone security technology," in *2020 European Conference on Circuit Theory and Design (ECCTD)*, 2020, pp. 1–4.